ĐẠI HỌC QUỐC GIA HÀ NỘI

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA TOÁN - CƠ - TIN HỌC

BÁO CÁO MÔN HỌC

ĐỀ TÀI NGHIÊN CỨU

TỐI ƯU HÓA ĐIỀU KHIỂN ĐÈN TÍN HIỆU GIAO THÔNG DỰA TRÊN MÔ HÌNH QUYẾT ĐỊNH MARKOV

GV hướng dẫn:

Nhóm sinh viên thực hiện:

Họ và tên	*MSSV*	*Mã lớp*
Nguyễn Quang Việt	22001659	K67A4
Phùng Hữu Uy	22001654	K67A4

Hà Nội, 2026

Mục lục

Lời cảm ơn **3**

Phân công nhiệm vụ **4**

1 Mở đầu **6**
1.1 Lý do chọn đề tài 6
1.2 Mục tiêu dự án 6
1.3 Phạm vi nghiên cứu 7
1.4 Cấu trúc báo cáo 7

2 Chương II: Cơ sở lý thuyết **9**
2.1 Mô hình Quyết định Markov (MDP) 9
2.1.1 Định nghĩa và thành phần 9
2.1.2 Tính chất Markov 9
2.1.3 Chính sách và mục tiêu tối ưu 10
2.1.4 Tại sao điều khiển đèn giao thông là một MDP? 10
2.2 Học tăng cường (Reinforcement Learning) 10
2.2.1 Khái niệm và đặc điểm 10
2.2.2 Khung tương tác Agent–Môi trường 11
2.2.3 Mục tiêu học tập 11
2.3 Q-Learning và Hàm giá trị hành động 11
2.3.1 Hàm giá trị hành động Q 11
2.3.2 Q-Learning truyền thống và hạn chế 12
2.4 Deep Q-Network (DQN) 12
2.4.1 Xấp xỉ hàm Q bằng mạng nơ-ron 12
2.4.2 Experience Replay 12
2.4.3 Target Network 13
2.4.4 Hàm mục tiêu và hàm mất mát 13
2.4.5 Chiến lược khám phá Epsilon-Greedy 13
2.5 Các cải tiến: Double DQN và Dueling Network 14
2.5.1 Double DQN — Giải quyết Overestimation Bias 14
2.5.2 Dueling Network Architecture 14

3 Chương III: Phương pháp đề xuất **15**
3.1 Tổng quan hệ thống 15
3.2 Môi trường mô phỏng SUMO 15
3.2.1 Phần mềm SUMO và TraCI 15
3.2.2 Kịch bản ngã tư Hà Nội mẫu 15

3.2.3 Phân bố phương tiện đặc trưng Việt Nam 16
3.3 Ánh xạ bài toán sang MDP 16
3.3.1 Không gian trạng thái 16
3.3.2 Không gian hành động 17
3.3.3 Hàm phần thưởng 17
3.3.4 Ràng buộc thời gian giữ pha 18
3.4 Kiến trúc mạng Deep Q-Network 18
3.4.1 Backbone MLP 18
3.4.2 Hai nhánh Dueling 19
3.5 Cấu hình huấn luyện 19
3.5.1 Siêu tham số 19
3.5.2 Bộ điều khiển baseline (Fixed-Time) 19
3.5.3 Vòng lặp huấn luyện tổng quát 20

4 Chương IV: Thực nghiệm và Đánh giá kết quả 21
4.1 Thiết lập thực nghiệm 21
4.1.1 Môi trường và cấu hình mô phỏng 21
4.1.2 Các chỉ số đánh giá 21
4.1.3 Phương pháp baseline 21
4.2 Kết quả so sánh DQN và Fixed-Time 22
4.2.1 Bảng so sánh tổng quan 22
4.2.2 Phân tích từng chỉ số 22
4.3 Phân tích cơ chế hoạt động của DQN Agent 23
4.3.1 Chiến lược thích ứng được học 23
4.3.2 So sánh đặc điểm hai chiến lược 23
4.4 Đánh giá tổng kết 24
4.4.1 Kết quả đạt được 24
4.4.2 Hạn chế và ghi nhận 24

5 Chương V: Kết luận 25
5.1 Tổng kết kết quả 25
5.2 Ưu điểm của phương pháp đề xuất 25
5.3 Hạn chế và hướng phát triển 26
5.3.1 Hạn chế hiện tại 26
5.3.2 Hướng phát triển tương lai 26

Tài liệu tham khảo 28

LỜI CẢM ƠN

Trong quá trình thực hiện đề tài *"Tối ưu hóa điều khiển đèn tín hiệu giao thông dựa trên Mô hình Quyết định Markov"* thuộc môn học Trí tuệ nhân tạo nâng cao, nhóm chúng em đã nhận được sự hỗ trợ và giúp đỡ quý báu từ nhiều phía. Những kiến thức lý thuyết về học tăng cường, mô hình quyết định Markov cùng với sự hướng dẫn tận tình của thầy cô đã trở thành nền tảng vững chắc để nhóm hoàn thành đề tài này.

Chúng em xin gửi lời cảm ơn chân thành đến giảng viên phụ trách môn học, người đã trực tiếp định hướng và luôn sẵn sàng hỗ trợ nhóm trong suốt quá trình nghiên cứu. Thầy đã dành thời gian hướng dẫn chi tiết từ việc lựa chọn phương pháp (Double DQN, Dueling Network) cho đến việc thiết kế môi trường mô phỏng SUMO phù hợp với đặc thù giao thông Việt Nam. Những buổi trao đổi với thầy không chỉ giúp chúng em nắm vững các khái niệm lý thuyết mà còn truyền cho chúng em niềm đam mê với bài toán tối ưu hóa trong thực tiễn.

Chúng em cũng xin cảm ơn các thầy cô trong khoa đã tạo điều kiện học tập tốt, truyền đạt những kiến thức quý giá về Trí tuệ nhân tạo và Học tăng cường — nền tảng trực tiếp cho đề tài này.

Trong quá trình thực hiện, hai thành viên trong nhóm đã luôn phối hợp chặt chẽ, hỗ trợ lẫn nhau trong từng giai đoạn từ xây dựng mô hình, cài đặt thuật toán, mô phỏng trên SUMO cho đến đánh giá và trực quan hóa kết quả. Chúng em hy vọng đề tài này sẽ đóng góp một phần nhỏ vào hướng nghiên cứu ứng dụng học tăng cường vào bài toán điều khiển giao thông thông minh tại Việt Nam.

Phân công nhiệm vụ

Dự án được thực hiện với sự tham gia của 2 thành viên, mỗi người đảm nhận các nhiệm vụ cụ thể như sau:

Nguyễn Quang Việt — Nhóm trưởng (MSSV: 22001659)

- Chịu trách nhiệm chính tổng thể dự án, phân công và điều phối công việc nhóm.
- Thiết kế kiến trúc hệ thống: xây dựng môi trường MDP (`SumoMDPEnv`) kết nối SUMO qua TraCI.
- Xây dựng và huấn luyện mô hình Double DQN kết hợp Dueling Network (`src/dqn/`).
- Thiết lập kịch bản mô phỏng ngã tư Hà Nội mẫu (file XML SUMO).
- Viết báo cáo: Chương 1 (Đặt vấn đề), Chương 2 (Cơ sở lý thuyết MDP và DQN), Chương 3 (Phương pháp đề xuất).
- Xây dựng script huấn luyện (`train.py`) và các script so sánh chiến lược.

Phùng Hữu Uy — Thành viên (MSSV: 22001654)

- Nghiên cứu và xây dựng bộ điều khiển thời gian cố định (`FixedTimeController`) làm baseline so sánh.
- Thiết kế hàm phần thưởng (reward function) phù hợp với bài toán giao thông Việt Nam.
- Xây dựng module đánh giá và trực quan hóa kết quả (`src/utils/plotting.py`).
- Thực hiện thực nghiệm so sánh DQN vs Fixed-Time và phân tích kết quả định lượng.
- Viết báo cáo: Chương 4 (Thực nghiệm và đánh giá kết quả), Chương 5 (Kết luận).
- Xây dựng các Jupyter Notebook đánh giá và trực quan hóa cho Chương 4.

Mỗi thành viên đều hỗ trợ lẫn nhau trong các giai đoạn quan trọng của dự án, bao gồm debug môi trường SUMO, tinh chỉnh tham số huấn luyện và hoàn thiện bản báo cáo, nhằm đảm bảo hoàn thành công việc đúng tiến độ và đạt chất lượng tốt nhất.

Họ và tên	**MSSV**	**Nhiệm vụ chính**
Nguyễn Quang Việt	22001659	Kiến trúc hệ thống, MDP, Double DQN, Chương 1–3
Phùng Hữu Uy	22001654	Baseline, Đánh giá, Trực quan hóa, Chương 4–5

1 Mở đầu

1.1 Lý do chọn đề tài

- Tình trạng ùn tắc giao thông tại các đô thị lớn của Việt Nam, đặc biệt là Hà Nội và TP. Hồ Chí Minh, đang ngày càng trở nên nghiêm trọng. Theo báo cáo của Ủy ban An toàn Giao thông Quốc gia, ùn tắc giao thông gây thiệt hại kinh tế ước tính hàng tỷ đồng mỗi ngày, đồng thời làm tăng mức độ ô nhiễm không khí và ảnh hưởng trực tiếp đến chất lượng cuộc sống của người dân.
- Hệ thống đèn tín hiệu giao thông hiện tại tại Việt Nam chủ yếu vẫn sử dụng phương pháp **điều khiển thời gian cố định** (Fixed-Time Control), trong đó mỗi pha đèn được cài đặt sẵn một khoảng thời gian không thay đổi bất kể mật độ giao thông thực tế. Phương pháp này đơn giản nhưng thiếu linh hoạt và không tối ưu trong các điều kiện giao thông biến động theo giờ, theo ngày hoặc theo sự kiện đặc biệt.
- Giao thông đô thị Việt Nam có đặc thù riêng so với các nước phát triển: tỷ lệ xe máy chiếm khoảng **60–70%** tổng lưu lượng phương tiện, cao hơn nhiều so với ô tô hay xe buýt. Sự đa dạng về loại phương tiện (xe máy, ô tô, xe buýt, xe tải) với kích thước và tốc độ khác nhau đặt ra yêu cầu đặc biệt khi xây dựng hệ thống điều khiển thông minh.
- Những năm gần đây, **Học tăng cường sâu** (Deep Reinforcement Learning — DRL) đã cho thấy hiệu quả vượt trội trong các bài toán điều khiển và tối ưu hóa phức tạp. Việc ứng dụng DRL, cụ thể là **Mô hình Quyết định Markov (MDP)** kết hợp với thuật toán **Deep Q-Network (DQN)**, vào bài toán điều khiển đèn tín hiệu mở ra hướng tiếp cận mới: hệ thống có thể *tự học* từ dữ liệu giao thông thực tế và liên tục điều chỉnh phương án điều khiển nhằm giảm thiểu thời gian chờ, hàng đợi và tắc nghẽn.
- Tuy nhiên, hầu hết các nghiên cứu hiện có về DRL cho điều khiển đèn giao thông đều được thực hiện trên các kịch bản giao thông châu Âu hoặc Bắc Mỹ, chưa phản ánh đúng đặc thù của giao thông Việt Nam. Vì vậy, **việc xây dựng một hệ thống điều khiển đèn thông minh được thiết kế riêng cho điều kiện giao thông Việt Nam** là cấp thiết và có ý nghĩa thực tiễn cao.

1.2 Mục tiêu dự án

Nhận thấy sự cấp thiết và tiềm năng của hướng nghiên cứu trên, nhóm đã xây dựng đề tài *"Tối ưu hóa điều khiển đèn tín hiệu giao thông dựa trên Mô hình Quyết định Markov"* với các mục tiêu cụ thể như sau:

- **Xây dựng môi trường mô phỏng giao thông** tại ngã tư đô thị Hà Nội mẫu sử dụng phần mềm SUMO (Simulation of Urban MObility), tích hợp phân bố phương tiện đặc trưng Việt Nam (60% xe máy, 30% ô tô, 10% xe buýt/xe tải) và hệ số quy đổi PCU (Passenger Car Unit) phù hợp thực tế.
- **Mô hình hóa bài toán điều khiển đèn tín hiệu** dưới dạng Mô hình Quyết định Markov (MDP), trong đó xác định rõ không gian trạng thái (hàng đợi, tốc độ, thời gian chờ, trọng số PCU theo làn), không gian hành động (4 pha đèn giao thông), và hàm phần thưởng phản ánh mức độ ùn tắc.
- **Triển khai thuật toán Double DQN kết hợp Dueling Network** để huấn luyện agent điều khiển đèn tự động, có khả năng thích ứng với biến động giao thông theo thời gian thực.
- **So sánh, đánh giá định lượng** hiệu quả của DQN agent so với phương pháp điều khiển thời gian cố định (baseline) thông qua các chỉ số: độ dài hàng đợi trung bình, thời gian chờ, tốc độ lưu thông và lưu lượng xe qua nút.

1.3 Phạm vi nghiên cứu

- **Đối tượng nghiên cứu:** Hệ thống điều khiển đèn tín hiệu tại nút giao thông 4 hướng (ngã tư) trong môi trường đô thị Việt Nam.
- **Công cụ mô phỏng:** Phần mềm SUMO kết hợp giao tiếp qua TraCI API.
- **Ngôn ngữ lập trình:** Python 3.9+ với framework PyTorch 2.2+.
- **Phạm vi thực nghiệm:** Kịch bản mô phỏng ngã tư mẫu với thời gian 1 giờ (3600 giây), huấn luyện 200.000 bước.

1.4 Cấu trúc báo cáo

Báo cáo được tổ chức thành các phần như sau:

- **Chương I — Mở đầu:** Lý do chọn đề tài, mục tiêu, phạm vi nghiên cứu.
- **Chương II — Cơ sở lý thuyết:** Mô hình Quyết định Markov, Học tăng cường, Deep Q-Network, Double DQN và Dueling Network.
- **Chương III — Phương pháp đề xuất:** Thiết kế môi trường MDP, kiến trúc mạng, hàm phần thưởng và các ràng buộc an toàn.
- **Chương IV — Thực nghiệm và Đánh giá:** Kết quả mô phỏng, so sánh DQN vs Fixed-Time, phân tích định lượng.

- **Chương V — Kết luận:** Tổng kết kết quả, hạn chế và hướng phát triển.

2 Chương II: Cơ sở lý thuyết

2.1 Mô hình Quyết định Markov (MDP)

2.1.1 Định nghĩa và thành phần

Mô hình Quyết định Markov (Markov Decision Process — MDP) là một khung toán học được sử dụng để mô hình hóa các bài toán ra quyết định tuần tự trong môi trường có tính ngẫu nhiên. MDP cung cấp nền tảng lý thuyết cho Học tăng cường (Reinforcement Learning), cho phép một tác nhân (agent) học cách tối ưu hóa hành vi của mình thông qua tương tác với môi trường.
Một MDP được định nghĩa bởi bộ 4 thành phần $\mathcal{M} = (S, A, P, R)$, trong đó:

- S: **Không gian trạng thái** (State Space) — tập hợp tất cả các trạng thái có thể có của môi trường. Tại mỗi thời điểm t, môi trường ở một trạng thái $s_t \in S$.
- A: **Không gian hành động** (Action Space) — tập hợp tất cả các hành động mà agent có thể thực hiện. Agent chọn hành động $a_t \in A$ tại mỗi bước.
- $P(s'|s, a)$: **Hàm chuyển trạng thái** (Transition Function) — xác suất chuyển từ trạng thái s sang trạng thái s' sau khi thực hiện hành động a:

$$P(s'|s, a) = \Pr(s_{t+1} = s' \mid s_t = s,\ a_t = a)$$

- $R(s, a)$: **Hàm phần thưởng** (Reward Function) — giá trị số thực đánh giá mức độ hiệu quả của hành động a tại trạng thái s. Phần thưởng tức thì $r_t = R(s_t, a_t)$ cung cấp tín hiệu học tập cho agent.

2.1.2 Tính chất Markov

Đặc trưng cốt lõi của MDP là **tính chất Markov**: trạng thái tương lai chỉ phụ thuộc vào trạng thái hiện tại và hành động hiện tại, không phụ thuộc vào lịch sử các trạng thái trước đó:

$$P(s_{t+1} \mid s_t, a_t) = P(s_{t+1} \mid s_0, a_0, s_1, a_1, \ldots, s_t, a_t)$$

Tính chất này cho phép đơn giản hóa đáng kể việc biểu diễn và giải quyết bài toán, vì agent chỉ cần quan sát trạng thái hiện tại để đưa ra quyết định tối ưu mà không cần ghi nhớ toàn bộ lịch sử.

2.1.3 Chính sách và mục tiêu tối ưu

Một **chính sách** (policy) $\pi : S \to A$ là ánh xạ từ trạng thái sang hành động, xác định cách agent hành xử trong mọi tình huống. Mục tiêu của MDP là tìm ra chính sách tối ưu π^* sao cho tối đa hóa **phần thưởng tích lũy chiết khấu kỳ vọng**:

$$\pi^* = \arg\max_{\pi} \; E_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right]$$

trong đó $\gamma \in (0, 1]$ là **hệ số chiết khấu** (discount factor). Giá trị γ nhỏ khiến agent ưu tiên phần thưởng ngắn hạn, trong khi γ gần 1 khuyến khích agent tối ưu hóa dài hạn. Trong dự án này, $\gamma = 0.99$, phản ánh tầm quan trọng của việc giảm ùn tắc bền vững trong suốt cả giờ mô phỏng.

2.1.4 Tại sao điều khiển đèn giao thông là một MDP?

Bài toán điều khiển đèn giao thông hội tụ đầy đủ các đặc trưng của MDP:

1. **Quyết định tuần tự:** Hành động chọn pha đèn tại bước t ảnh hưởng trực tiếp đến trạng thái giao thông tại $t + 1$.

2. **Tính ngẫu nhiên:** Phương tiện đến ngẫu nhiên, không thể dự đoán chính xác trạng thái tiếp theo.

3. **Tối ưu dài hạn:** Mục tiêu không chỉ là giảm ùn tắc ngay lập tức mà cả trong tương lai.

4. **Không biết trước mô hình:** Hàm chuyển trạng thái $P(s'|s, a)$ không được biết tường minh, cần học từ dữ liệu.

2.2 Học tăng cường (Reinforcement Learning)

2.2.1 Khái niệm và đặc điểm

Học tăng cường (Reinforcement Learning — RL) là một nhánh của học máy, trong đó một agent học cách ra quyết định tối ưu thông qua quá trình tương tác thử-sai (trial-and-error) với môi trường. Không giống như Học có giám sát (Supervised Learning), RL không yêu cầu dữ liệu được gán nhãn trước; thay vào đó, agent nhận tín hiệu phản hồi dưới dạng phần thưởng từ môi trường. Những đặc điểm nổi bật của RL:

- Không có nhãn đúng–sai tường minh như Học có giám sát.

- Phần thưởng có thể bị trễ theo thời gian (delayed reward) — hành động hiện tại có thể chỉ được đánh giá sau nhiều bước.

- Mục tiêu là tối ưu hóa hiệu quả dài hạn, không chỉ phần thưởng tức thì.
- Phù hợp với các bài toán điều khiển và ra quyết định trong môi trường động.

2.2.2 Khung tương tác Agent–Môi trường

Khung RL chuẩn bao gồm các thành phần:

- **Agent**: Tác nhân ra quyết định, tương ứng với hệ thống điều khiển đèn.
- **Environment**: Môi trường tương tác, tương ứng với nút giao thông và luồng xe.
- **State** (s_t): Trạng thái quan sát được của môi trường tại thời điểm t.
- **Action** (a_t): Hành động agent thực hiện (chọn pha đèn).
- **Reward** (r_t): Phản hồi từ môi trường sau mỗi hành động.

Tại mỗi bước thời gian t, vòng lặp tương tác diễn ra như sau:

1. Agent quan sát trạng thái hiện tại s_t.
2. Agent chọn hành động a_t theo chính sách π.
3. Môi trường chuyển sang trạng thái s_{t+1} và trả về phần thưởng r_{t+1}.
4. Agent cập nhật kiến thức và lặp lại.

2.2.3 Mục tiêu học tập

Agent cần học chính sách π để tối đa hóa tổng phần thưởng tích lũy chiết khấu từ bước t:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

Đây chính là **return** (lợi tức), thể hiện giá trị lâu dài của việc theo một chính sách nhất định từ trạng thái s_t.

2.3 Q-Learning và Hàm giá trị hành động

2.3.1 Hàm giá trị hành động Q

Hàm giá trị hành động $Q^\pi(s, a)$ biểu thị tổng phần thưởng kỳ vọng khi thực hiện hành động a tại trạng thái s và sau đó tuân theo chính sách π:

$$Q^\pi(s, a) = E_\pi \left[G_t \mid s_t = s,\ a_t = a \right]$$

Hàm Q tối ưu $Q^*(s, a)$ thỏa mãn **phương trình Bellman tối ưu**:

$$Q^*(s, a) = E\left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a\right]$$

Từ Q^*, chính sách tối ưu được xác định đơn giản bằng:

$$\pi^*(s) = \arg\max_a Q^*(s, a)$$

2.3.2 Q-Learning truyền thống và hạn chế

Q-Learning cập nhật bảng Q theo quy tắc:

$$Q(s, a) \leftarrow Q(s, a) + \alpha\left[r + \gamma \max_{a'} Q(s', a') - Q(s, a)\right]$$

Tuy nhiên, phương pháp bảng Q (Q-table) có hạn chế nghiêm trọng: không gian trạng thái cần được rời rạc hóa và có kích thước hữu hạn. Trong bài toán điều khiển đèn giao thông, không gian trạng thái là liên tục và có số chiều cao (32 chiều với 8 làn), khiến Q-table trở nên bất khả thi. Đây là động lực để sử dụng mạng nơ-ron sâu.

2.4 Deep Q-Network (DQN)

2.4.1 Xấp xỉ hàm Q bằng mạng nơ-ron

Deep Q-Network (DQN) thay thế bảng Q bằng một mạng nơ-ron sâu $Q(s, a; \theta)$ tham số hóa bởi θ để xấp xỉ hàm giá trị hành động:

$$Q(s, a) \approx Q(s, a;\, \theta)$$

Mạng nhận đầu vào là vector trạng thái s và đầu ra là giá trị Q cho tất cả các hành động, cho phép xử lý không gian trạng thái liên tục và có số chiều lớn.

2.4.2 Experience Replay

Một trong hai kỹ thuật ổn định huấn luyện cốt lõi của DQN là **Experience Replay**. Các transition (s, a, r, s', d) được lưu vào **replay buffer** (bộ nhớ kinh nghiệm) dung lượng N. Thay vì học trực tiếp từ dữ liệu liên tiếp, agent lấy mẫu ngẫu nhiên một *minibatch* từ buffer để cập nhật mạng.
Kỹ thuật này mang lại hai lợi ích:

- **Giảm tương quan**: Dữ liệu liên tiếp trong RL thường có tương quan cao, gây mất ổn định khi huấn luyện. Lấy mẫu ngẫu nhiên phá vỡ tương quan này.
- **Tái sử dụng dữ liệu**: Mỗi transition có thể được học nhiều lần, tăng hiệu quả sử dụng dữ liệu.

2.4.3 Target Network

Kỹ thuật ổn định thứ hai là sử dụng **mạng target** (target network) $Q(s, a; \theta^-)$ tách biệt với mạng online $Q(s, a; \theta)$. Mạng target được sử dụng để tính TD target:

$$y = r + \gamma \max_{a'} Q(s', a';\, \theta^-)$$

Tham số θ^- được cập nhật định kỳ (hard update) bằng cách sao chép từ θ mỗi N_{update} bước, thay vì cập nhật liên tục. Điều này ngăn chặn hiện tượng mục tiêu học tập bị dao động liên tục (*moving target problem*), giúp ổn định quá trình huấn luyện.

2.4.4 Hàm mục tiêu và hàm mất mát

DQN tối thiểu hóa sai lệch giữa giá trị Q hiện tại và TD target. Thay vì dùng MSE (Mean Squared Error), DQN sử dụng **Huber Loss** (Smooth L1 Loss) để giảm độ nhạy cảm với các giá trị ngoại lai (outlier):

$$\mathcal{L}(\theta) = E_{(s,a,r,s')\sim\mathcal{B}}\left[\text{SmoothL1}(Q(s, a; \theta) - y)\right]$$

trong đó:

$$\text{SmoothL1}(x) = \begin{cases} \frac{1}{2}x^2 & \text{nếu } |x| \leq 1 \\ |x| - \frac{1}{2} & \text{nếu } |x| > 1 \end{cases}$$

Ngoài ra, **gradient clipping** (giới hạn chuẩn gradient tối đa bằng 10) được áp dụng để ổn định bước cập nhật tham số.

2.4.5 Chiến lược khám phá Epsilon-Greedy

Để cân bằng giữa **khai thác** (exploitation — sử dụng kiến thức đã học) và **khám phá** (exploration — thử nghiệm hành động mới), DQN sử dụng chiến lược ϵ-greedy:

$$a_t = \begin{cases} \text{hành động ngẫu nhiên} & \text{với xác suất } \epsilon \\ \arg\max_a Q(s_t, a; \theta) & \text{với xác suất } 1 - \epsilon \end{cases}$$

Trong dự án này, ϵ giảm tuyến tính từ $\epsilon_{\text{start}} = 1.0$ xuống $\epsilon_{\text{end}} = 0.05$ trong suốt $T = 100.000$ bước đầu:

$$\epsilon(t) = \epsilon_{\text{start}} + \frac{\min(t,\, T)}{T} \cdot (\epsilon_{\text{end}} - \epsilon_{\text{start}})$$

Giai đoạn đầu (nhiều exploration) giúp agent thu thập đa dạng kinh nghiệm; giai đoạn sau (nhiều exploitation) giúp agent tinh chỉnh chính sách đã học.

2.5 Các cải tiến: Double DQN và Dueling Network

2.5.1 Double DQN — Giải quyết Overestimation Bias

DQN tiêu chuẩn có xu hướng **ước lượng quá cao** (overestimate) giá trị Q do sử dụng cùng một mạng để vừa chọn hành động vừa đánh giá giá trị của hành động đó:

$$y_{\text{DQN}} = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$

Double DQN giải quyết vấn đề này bằng cách *tách biệt* hai vai trò:

- **Mạng online** $Q(\cdot; \theta)$: chọn hành động tốt nhất tại s'.
- **Mạng target** $Q(\cdot; \theta^-)$: đánh giá giá trị của hành động được chọn.

$$a^* = \arg\max_{a'} Q(s', a'; \theta)$$

$$y_{\text{Double}} = r + \gamma\, Q(s', a^*; \theta^-)$$

Sự tách biệt này làm giảm overestimation bias, dẫn đến ước lượng Q chính xác hơn và chính sách ổn định hơn.

2.5.2 Dueling Network Architecture

Kiến trúc Dueling (Dueling DQN) tái cấu trúc mạng nơ-ron để tường minh tách biệt hai thành phần của hàm Q:

- **Value stream** $V(s; \theta_v)$: ước lượng giá trị của trạng thái s bất kể hành động nào được chọn.
- **Advantage stream** $A(s, a; \theta_a)$: ước lượng lợi thế tương đối của hành động a so với giá trị trung bình của các hành động tại trạng thái s.

Hai luồng được kết hợp theo công thức:

$$Q(s, a; \theta) = V(s; \theta_v) + \left[A(s, a; \theta_a) - \frac{1}{|A|} \sum_{a'} A(s, a'; \theta_a) \right]$$

Việc trừ đi giá trị trung bình của advantage đảm bảo tính duy nhất (identifiability) của V và A.
Ưu điểm của Dueling Architecture: trong nhiều tình huống giao thông, giá trị của trạng thái (mức độ ùn tắc) quan trọng hơn việc phân biệt giữa các hành động. Dueling cho phép mạng học $V(s)$ một cách hiệu quả ngay cả khi không cần phân biệt tất cả các hành động, từ đó tăng tốc và ổn định quá trình học.

3 Chương III: Phương pháp đề xuất

3.1 Tổng quan hệ thống

Hệ thống được đề xuất gồm ba lớp chính tương tác với nhau:

1. **Lớp môi trường mô phỏng** (SUMO): tái tạo ngã tư đô thị với luồng phương tiện đặc trưng Việt Nam.
2. **Lớp ánh xạ MDP**: chuyển đổi dữ liệu từ SUMO thành không gian trạng thái, hành động và phần thưởng theo chuẩn MDP.
3. **Lớp agent DQN**: mạng nơ-ron Deep Q-Network nhận trạng thái, ra quyết định chọn pha đèn và cập nhật trọng số qua quá trình học.

Vòng lặp hoạt động tổng quát: tại mỗi bước, agent quan sát trạng thái s_t từ SUMO thông qua TraCI API, chọn hành động a_t (pha đèn), thực thi pha đó trong 5 giây mô phỏng, nhận phần thưởng r_t và trạng thái mới s_{t+1}, rồi lưu transition vào replay buffer và cập nhật mạng.

3.2 Môi trường mô phỏng SUMO

3.2.1 Phần mềm SUMO và TraCI

SUMO (Simulation of Urban MObility) là phần mềm mô phỏng giao thông đô thị mã nguồn mở, cho phép mô hình hóa chi tiết chuyển động của từng phương tiện, hành vi tại nút giao và hoạt động của đèn tín hiệu. Giao tiếp giữa Python và SUMO được thực hiện qua **TraCI** (Traffic Control Interface) API, cho phép:

- Đọc trạng thái giao thông theo thời gian thực (số xe dừng, tốc độ, thời gian chờ...).
- Điều khiển pha đèn tín hiệu từ bên ngoài (`setPhase`, `setPhaseDuration`).
- Tiến hành từng bước mô phỏng (`simulationStep`).

3.2.2 Kịch bản ngã tư Hà Nội mẫu

Kịch bản được xây dựng mô phỏng một ngã tư 4 hướng đặc trưng tại đô thị Hà Nội, gồm:

- **Cấu trúc mạng lưới**: Nút trung tâm `c` (có đèn tín hiệu) kết nối 4 hướng Bắc (`n`), Nam (`s`), Đông (`e`), Tây (`w`). Mỗi hướng có 2 làn xe vào và 2 làn xe ra, tốc độ tối đa 50 km/h ($\approx$ 13.9 m/s).

- **Thời gian mô phỏng**: 3600 giây (1 giờ), đại diện cho một khung giờ cao điểm.
- **Giai đoạn khởi động** (warmup): 60 giây đầu chạy mô phỏng không can thiệp để tạo mật độ giao thông ban đầu ổn định trước khi agent bắt đầu điều khiển.

3.2.3 Phân bố phương tiện đặc trưng Việt Nam

Kịch bản tích hợp 4 loại phương tiện phản ánh thực tế giao thông Việt Nam:

Loại xe	**Tỉ lệ (%)**	**Chiều dài (m)**	**Tốc độ tối đa (m/s)**	**Hệ số PC**
Xe máy (motorcycle)	∼60	2.0	16.67	0.5
Ô tô (car)	∼30	5.0	13.89	1.5
Xe buýt (bus)	∼5	12.0	11.11	2.0
Xe tải (truck)	∼5	7.5	11.11	2.0

Hệ số PCU (Passenger Car Unit) được chuẩn hóa theo thực tế Việt Nam: 1 ô tô tương đương 3 xe máy về chiếm dụng diện tích và ảnh hưởng đến năng lực thông hành.

3.3 Ánh xạ bài toán sang MDP

3.3.1 Không gian trạng thái

Trạng thái s_t là vector liên tục thu thập từ tất cả các làn xe vào nút, trong đó mỗi làn được biểu diễn bởi 4 đặc trưng:

- q_i: **Số xe dừng** (queue length) trên làn i — số phương tiện có tốc độ gần bằng 0.
- v_i: **Tốc độ trung bình** (average speed) trên làn i tính bằng m/s.
- w_i: **Thời gian chờ tích lũy** (waiting time) trên làn i tính bằng giây.
- c_i: **Số xe quy đổi PCU** (weighted vehicle count) trên làn i — tổng trọng số PCU của tất cả xe đang có mặt.

Với $L = 8$ làn (4 hướng $\times$ 2 làn/hướng), vector trạng thái có số chiều:

$$s_t \in R^{4L} = R^{32}$$

Trọng số PCU theo loại xe:

$$c_i = \sum_{v \in \text{lane}_i} \begin{cases} 0.5 & \text{nếu } v \text{ là xe máy} \\ 1.5 & \text{nếu } v \text{ là ô tô} \\ 2.0 & \text{nếu } v \text{ là xe buýt hoặc xe tải} \end{cases}$$

3.3.2 Không gian hành động

Agent có 4 hành động tương ứng với 4 pha tín hiệu được định nghĩa trước:

Pha	Hướng được phép	Mô tả
a_0	Bắc–Nam **xanh**	Xe đi thẳng và rẽ phải hướng Bắc/Nam
a_1	Bắc–Nam **xanh trái**	Xe rẽ trái hướng Bắc/Nam
a_2	Đông–Tây **xanh**	Xe đi thẳng và rẽ phải hướng Đông/Tây
a_3	Đông–Tây **xanh trái**	Xe rẽ trái hướng Đông/Tây

Agent chọn một hành động mỗi **5 giây** mô phỏng (`action_duration = 5s`). Tại mỗi bước, SUMO tiến thêm 5 bước giây (`simulationStep`) với pha đèn được giữ nguyên.

3.3.3 Hàm phần thưởng

Hàm phần thưởng được thiết kế để phạt trực tiếp tình trạng ùn tắc:

$$R_t = -\left(Q_t + \frac{W_t}{60}\right) + \text{penalty}$$

trong đó:

- $Q_t = \sum_{i=1}^{L} q_i^{(t)}$: tổng số xe dừng trên tất cả các làn tại bước t.
- $W_t = \sum_{i=1}^{L} w_i^{(t)}$: tổng thời gian chờ (giây) trên tất cả các làn, chia 60 để chuẩn hóa về cùng đơn vị với Q_t.
- penalty: phạt thêm -1.0 nếu agent cố chuyển pha trước khi hết thời gian giữ tối thiểu.

Phần thưởng R_t luôn âm; giá trị càng gần 0 biểu thị giao thông càng thông thoáng. Mục tiêu của agent là tối đa hóa tổng phần thưởng, tương đương với việc tối thiểu hóa hàng đợi và thời gian chờ.

3.3.4 Ràng buộc thời gian giữ pha

Để đảm bảo an toàn giao thông và ổn định quá trình học, hệ thống áp đặt ba ràng buộc về thời gian giữ pha:

- **Thời gian giữ tối thiểu** $t_{\min} = 30$ giây (tương đương 6 bước với $\Delta t = 5$s):
  - Nếu agent muốn chuyển pha nhưng pha hiện tại chưa được giữ đủ 30 giây, hệ thống *bác bỏ hành động*, tiếp tục giữ pha cũ và áp dụng mức phạt -1.0 vào phần thưởng.
  - Ý nghĩa: ngăn tình huống đèn xanh chỉ kéo dài vài giây, gây nguy hiểm cho phương tiện đang đi qua giao lộ.
- **Thời gian giữ tối đa** $t_{\max} = 120$ giây (tương đương 24 bước):
  - Nếu một pha đã được giữ quá 120 giây, hệ thống *bắt buộc chuyển* sang pha kế tiếp trong chu kỳ, bất kể quyết định của agent.
  - Ý nghĩa: ngăn tình trạng một hướng bị kẹt đỏ vô thời hạn, đảm bảo công bằng cho tất cả các hướng.
- **Phạt chuyển sớm**: Mỗi lần agent cố chuyển pha trước $t_{\min}$, phần thưởng bị trừ thêm -1.0, khuyến khích agent học được tính kiên nhẫn giữ pha đủ thời gian.

3.4 Kiến trúc mạng Deep Q-Network

3.4.1 Backbone MLP

Mạng backbone gồm 2 lớp ẩn kết nối đầy đủ (fully connected), mỗi lớp 128 neuron:

$$\text{Input}(32) \xrightarrow{\text{FC+LN+ReLU}} 128 \xrightarrow{\text{FC+LN+ReLU}} 128$$

Mỗi lớp ẩn sử dụng tổ hợp:

- **Linear** (kết nối tuyến tính),
- **LayerNorm** (chuẩn hóa lớp) — ổn định phân phối kích hoạt theo chiều đặc trưng,
- **ReLU** (hàm kích hoạt phi tuyến).

3.4.2 Hai nhánh Dueling

Sau backbone, mạng tách thành hai nhánh song song:
Value stream (ước lượng giá trị trạng thái):

$$V(s;\theta_v) : 128 \xrightarrow{\text{FC+ReLU}} 128 \xrightarrow{\text{FC}} 1$$

Advantage stream (ước lượng lợi thế hành động):

$$A(s,a;\theta_a) : 128 \xrightarrow{\text{FC+ReLU}} 128 \xrightarrow{\text{FC}} 4$$

Đầu ra Q được tổng hợp:

$$Q(s,a;\theta) = V(s;\theta_v) + A(s,a;\theta_a) - \frac{1}{4}\sum_{a'=0}^{3} A(s,a';\theta_a)$$

Toàn bộ tham số được khởi tạo bằng **Kaiming Uniform** để tránh vanishing/exploding gradient khi huấn luyện sâu.

3.5 Cấu hình huấn luyện

3.5.1 Siêu tham số

Tham số	**Giá trị**	**Mô tả**
Tổng bước huấn luyện	200.000	Số bước tương tác agent–môi trường
Bước bắt đầu train	10.000	Thu thập kinh nghiệm trước khi học
Kích thước replay buffer	100.000	Số transition lưu trong bộ nhớ
Kích thước batch	64	Số mẫu mỗi lần cập nhật mạng
Hệ số chiết khấu γ	0.99	Tầm quan trọng phần thưởng dài hạn
Learning rate α	0.0005	Tốc độ học của Adam optimizer
Cập nhật target network	mỗi 2.000 bước	Hard copy tham số
Gradient clipping	10.0	Giới hạn chuẩn gradient
ϵ_{start}	1.0	Epsilon khởi đầu (toàn khám phá)
ϵ_{end}	0.05	Epsilon cuối (5% khám phá)
Số bước giảm ϵ	100.000	Giảm tuyến tính trong 100k bước đầu

3.5.2 Bộ điều khiển baseline (Fixed-Time)

Để so sánh, nhóm xây dựng bộ điều khiển thời gian cố định với cấu hình:

- Chu kỳ đèn: 4 pha $\times$ (55 giây xanh + 5 giây vàng) = 240 giây.
- Luân chuyển pha theo thứ tự cố định $a_0 \to a_1 \to a_2 \to a_3 \to a_0 \to \cdots$
- Không có khả năng thích ứng với tình trạng giao thông thực tế.

3.5.3 Vòng lặp huấn luyện tổng quát

1. Khởi tạo môi trường SUMO, agent DQN, replay buffer và lịch giảm ϵ.
2. Lặp lại trong $T = 200.000$ bước:
   (a) Tính ϵ_t theo lịch giảm tuyến tính.
   (b) Agent chọn hành động a_t theo ϵ-greedy từ trạng thái s_t.
   (c) Thực thi a_t: SUMO tiến 5 bước giây và trả về $(r_t, s_{t+1}, \text{done})$.
   (d) Lưu transition $(s_t, a_t, r_t, s_{t+1}, \text{done})$ vào replay buffer.
   (e) Nếu buffer có đủ ≥ 10.000 mẫu: lấy batch 64 mẫu, tính loss Double DQN, cập nhật θ bằng Adam.
   (f) Mỗi 2.000 bước: cập nhật target network $\theta^- \leftarrow \theta$.
   (g) Nếu episode kết thúc (sau 3600 giây): reset môi trường SUMO.
3. Lưu trọng số model tốt nhất vào `outputs/dqn_vn_tls.pt`.

4 Chương IV: Thực nghiệm và Đánh giá kết quả

4.1 Thiết lập thực nghiệm

4.1.1 Môi trường và cấu hình mô phỏng

Toàn bộ thực nghiệm được thực hiện trên kịch bản ngã tư mẫu Hà Nội trong phần mềm SUMO, với các tham số cấu hình như sau:

Tham số	**Giá trị**
Kịch bản	Ngã tư 4 hướng, Hà Nội mẫu
Thời gian bước mô phỏng	1.0 giây
Thời gian mỗi episode	3600 bước (1 giờ)
Thời gian khởi động (warmup)	60 bước
Khoảng thời gian mỗi hành động	5 giây
Chế độ GUI	Tắt (headless)

4.1.2 Các chỉ số đánh giá

Hiệu quả của hai chiến lược điều khiển được đánh giá qua 5 chỉ số chính:

- **Độ dài hàng đợi trung bình** (Avg Queue Length): số xe dừng trung bình trên tất cả các làn trong suốt episode. Chỉ số này phản ánh trực tiếp mức độ ùn tắc.
- **Độ dài hàng đợi cực đại** (Max Queue Length): số xe dừng lớn nhất ghi nhận tại bất kỳ thời điểm nào, đo lường khả năng xử lý đỉnh ùn tắc.
- **Thời gian chờ trung bình** (Avg Wait Time): thời gian chờ trung bình (giây) mỗi bước đo, phản ánh mức độ khó chịu của người tham gia giao thông.
- **Lưu lượng xe qua nút** (Throughput): tổng số xe rời khỏi khu vực mô phỏng thành công trong một episode.
- **Tổng phần thưởng** (Total Reward): tổng phần thưởng $\sum_t R_t$ tích lũy của agent trong một episode, là chỉ số tổng hợp trực tiếp của hàm mục tiêu.

4.1.3 Phương pháp baseline

Bộ điều khiển thời gian cố định (Fixed-Time) được cấu hình với chu kỳ 120 giây:

Tham số	Giá trị
Chiến lược	Luân chuyển pha cố định
Số pha trong chu kỳ	4
Thời gian xanh mỗi pha	25 giây
Thời gian vàng mỗi pha	5 giây
Tổng chu kỳ	120 giây
Khả năng thích ứng	Không có

4.2 Kết quả so sánh DQN và Fixed-Time

4.2.1 Bảng so sánh tổng quan

Bảng dưới đây tổng hợp kết quả trung bình từ thực nghiệm so sánh song song giữa hai chiến lược điều khiển trên cùng kịch bản giao thông:

Chỉ số	DQN	Fixed-Time	Chênh lệch	Tốt hơn
Hàng đợi trung bình (xe)	5.18	31.54	−26.36	DQN
Hàng đợi cực đại (xe)	12.00	45.00	−33.00	DQN
Độ lệch chuẩn hàng đợi	1.65	8.45	−6.80	DQN
Thời gian chờ TB (s)	2.25	251.15	−248.89	DQN
Lưu lượng xe (xe/ep.)	274	250	+24	DQN
Tổng phần thưởng	−3.835	−34.764	+30.929	DQN

4.2.2 Phân tích từng chỉ số

(1) Độ dài hàng đợi:
DQN duy trì hàng đợi trung bình chỉ **5.18 xe/làn**, so với **31.54 xe/làn** của Fixed-Time — tương đương mức giảm **83.6%**. Đặc biệt đáng chú ý là hàng đợi cực đại của DQN chỉ đạt 12 xe trong khi Fixed-Time lên đến 45 xe, tức DQN kiểm soát được các đỉnh ùn tắc tốt hơn **3.75 lần**.
Ngoài ra, độ lệch chuẩn hàng đợi của DQN (1.65) thấp hơn Fixed-Time (8.45) gần **5 lần**, cho thấy agent học được chính sách điều khiển *ổn định và nhất quán* theo thời gian, thay vì để xảy ra các đợt ùn tắc cục bộ như điều khiển cố định.
(2) Thời gian chờ:
Thời gian chờ trung bình của DQN là **2.25 giây/bước**, giảm **99.1%** so với Fixed-Time (**251.15 giây/bước**). Đây là cải thiện nổi bật nhất, phản ánh rằng agent DQN đã học được cách phân bổ pha xanh đúng lúc và đúng hướng, tránh để xe dừng tích lũy thời gian chờ dài.
Kết quả từ thực nghiệm độc lập trên bộ dữ liệu Chương 4 (2 episode) cũng xác nhận kết quả này: DQN đạt thời gian chờ trung bình **8.46 giây** trong khi Fixed-Time là **2243.91 giây** — mức chênh lệch hơn **265 lần**.
(3) Lưu lượng xe (Throughput):

DQN cho phép **274 xe** qua nút trong một episode so với **250 xe** của Fixed-Time, tăng **9.6%**. Điều này cho thấy DQN không chỉ giảm ùn tắc mà còn cải thiện năng lực thông hành của nút giao, cho phép nhiều xe hoàn thành hành trình hơn trong cùng thời gian.

(4) Tổng phần thưởng:

Tổng phần thưởng của DQN đạt **−3,835** so với **−34,764** của Fixed-Time. Vì phần thưởng được định nghĩa là giá trị âm tỉ lệ với mức độ ùn tắc, phần thưởng DQN cao hơn **89%** khẳng định rằng agent đã tối ưu hóa thành công hàm mục tiêu đề ra.

4.3 Phân tích cơ chế hoạt động của DQN Agent

4.3.1 Chiến lược thích ứng được học

Qua quá trình huấn luyện 200.000 bước, agent DQN học được các hành vi thích ứng sau:

- **Ưu tiên làn có hàng đợi dài:** Agent học cách quan sát đặc trưng hàng đợi q_i và tập trung thời gian xanh cho các hướng đang tích lũy xe, thay vì luân chuyển đều đặn theo chu kỳ cố định.
- **Giữ pha đủ lâu:** Nhờ ràng buộc $t_{\min} = 30$s và phạt chuyển sớm, agent học được hành vi giữ pha đủ thời gian để xe có thể qua giao lộ, tránh tình trạng chuyển pha liên tục gây mất hiệu quả.
- **Không giữ pha quá lâu:** Ràng buộc $t_{\max} = 120$s đảm bảo agent không "bỏ quên" các hướng khác trong thời gian dài, giúp phân bổ thời gian xanh công bằng hơn so với Fixed-Time.

4.3.2 So sánh đặc điểm hai chiến lược

Đặc điểm	**Fixed-Time**	**DQN**
Khả năng thích ứng	Không có	Theo thời gian thực
Nhận thức trạng thái giao thông	Không	Có (vector 32 chiều)
Khả năng học	Không	Có (RL)
Tối ưu hóa	Xác định trước	Dựa trên kinh nghiệm
Chi phí tính toán	Rất thấp	Vừa phải
Độ phức tạp triển khai	Đơn giản	Phức tạp hơn

4.4 Đánh giá tổng kết

4.4.1 Kết quả đạt được

Thực nghiệm cho thấy hệ thống điều khiển đèn dựa trên Double DQN + Dueling Network vượt trội hơn phương pháp điều khiển thời gian cố định trên hầu hết các chỉ số quan trọng:

- Giảm hàng đợi trung bình **83.6%** (5.18 xe so với 31.54 xe).
- Giảm thời gian chờ trung bình **99.1%** (2.25s so với 251.15s).
- Tăng lưu lượng xe qua nút **9.6%** (274 xe so với 250 xe).
- Cải thiện tổng phần thưởng **89%** ($-3{,}835$ so với $-34{,}764$).
- Hàng đợi ổn định hơn (độ lệch chuẩn thấp hơn 5 lần).

4.4.2 Hạn chế và ghi nhận

Bên cạnh các kết quả tích cực, thực nghiệm cũng ghi nhận một số hạn chế cần lưu ý:

- **Tốc độ trung bình:** Trong kịch bản so sánh song song, tốc độ trung bình của DQN (1.97 m/s) thấp hơn Fixed-Time (2.44 m/s). Điều này có thể do DQN giữ nhiều xe trong trạng thái dừng chờ ở một số pha, làm giảm tốc độ tổng thể dù hàng đợi thực tế thấp hơn. Đây là điểm cần cải thiện trong các phiên bản tiếp theo.
- **Phạm vi thực nghiệm:** Thực nghiệm chỉ được thực hiện trên 1 ngã tư đơn với lưu lượng giao thông được mô phỏng theo xác suất cố định. Kết quả có thể thay đổi khi áp dụng cho các kịch bản phức tạp hơn (giờ cao điểm, mạng lưới nhiều nút).
- **Thời gian huấn luyện:** Quá trình huấn luyện 200.000 bước đòi hỏi khoảng 30–60 phút tùy cấu hình máy tính, chưa phù hợp cho triển khai online thời gian thực.
- **Chưa triển khai thực tế:** Kết quả hiện tại chỉ được kiểm chứng trong môi trường mô phỏng SUMO; cần thêm bước xác thực trên dữ liệu giao thông thực trước khi có thể triển khai tại các nút giao thực tế.

5 Chương V: Kết luận

5.1 Tổng kết kết quả

Đề tài *"Tối ưu hóa điều khiển đèn tín hiệu giao thông dựa trên Mô hình Quyết định Markov"* đã được xây dựng và thực nghiệm hoàn chỉnh với các kết quả đáng khích lệ.
Nhóm đã thành công trong việc mô hình hóa bài toán điều khiển đèn tín hiệu tại một ngã tư đô thị Việt Nam dưới dạng Mô hình Quyết định Markov (MDP), triển khai thuật toán **Double DQN kết hợp Dueling Network** và huấn luyện agent trong môi trường mô phỏng SUMO với các đặc điểm giao thông đặc trưng của Việt Nam (60% xe máy, hệ số PCU chuẩn hóa theo thực tế).
Kết quả thực nghiệm so sánh với phương pháp điều khiển thời gian cố định (Fixed-Time) cho thấy hệ thống DQN đề xuất vượt trội ở hầu hết các chỉ số quan trọng:

- **Giảm hàng đợi trung bình 83.6%**: từ 31.54 xe/làn (Fixed-Time) xuống còn 5.18 xe/làn (DQN).
- **Giảm thời gian chờ 99.1%**: từ 251.15 giây xuống còn 2.25 giây.
- **Tăng lưu lượng xe 9.6%**: từ 250 xe/episode lên 274 xe/episode.
- **Cải thiện tổng phần thưởng 89%**: từ $-34{,}764$ lên $-3{,}835$.
- **Ổn định hơn 5 lần**: độ lệch chuẩn hàng đợi giảm từ 8.45 xuống 1.65.

Những kết quả này chứng minh rằng Học tăng cường sâu hoàn toàn có thể ứng dụng hiệu quả vào bài toán điều khiển đèn giao thông, mang lại cải thiện đáng kể so với phương pháp truyền thống. Đặc biệt, việc tích hợp đặc thù giao thông Việt Nam (trọng số PCU xe máy, phân bố phương tiện thực tế) vào thiết kế môi trường MDP là bước tiếp cận phù hợp và cần thiết.

5.2 Ưu điểm của phương pháp đề xuất

- **Khả năng thích ứng theo thời gian thực:** Agent quan sát trạng thái giao thông liên tục và điều chỉnh pha đèn dựa trên tình huống thực tế, khác với lịch cố định không nhận biết được mật độ xe.
- **Không cần mô hình vật lý:** Agent học trực tiếp từ tương tác với môi trường mô phỏng mà không cần biết tường minh hàm chuyển trạng thái $P(s'|s,a)$ — một ưu điểm lớn so với các phương pháp điều khiển tối ưu truyền thống.

- **Có thể mở rộng:** Kiến trúc và thuật toán đã xây dựng có tiềm năng mở rộng lên các kịch bản phức tạp hơn như mạng lưới nhiều nút giao, điều phối đa tác nhân (multi-agent) hoặc tích hợp dữ liệu cảm biến thực tế.
- **Thiết kế dành riêng cho Việt Nam:** Hệ số PCU theo chuẩn Việt Nam, phân bố xe máy chiếm ưu thế và các ràng buộc an toàn thực tế được tích hợp trực tiếp vào thiết kế, tạo nên điểm khác biệt so với các nghiên cứu quốc tế.

5.3 Hạn chế và hướng phát triển

5.3.1 Hạn chế hiện tại

- **Phạm vi thực nghiệm còn hẹp:** Dự án chỉ kiểm thử trên một ngã tư đơn với lưu lượng giao thông được tạo theo xác suất ngẫu nhiên. Kết quả cần được xác nhận lại trên dữ liệu giao thông thực tế hoặc các kịch bản phức tạp hơn (nút T, vòng xuyến, mạng lưới nhiều nút).
- **Thời gian huấn luyện:** Quá trình huấn luyện 200.000 bước yêu cầu 30–60 phút tính toán. Điều này chưa phù hợp cho các ứng dụng cần học online và thích ứng ngay lập tức với thay đổi điều kiện giao thông.
- **Chưa triển khai thực tế:** Toàn bộ kết quả được kiểm chứng trong môi trường mô phỏng SUMO. Khoảng cách giữa mô phỏng và thực tế (sim-to-real gap) cần được đánh giá kỹ lưỡng trước khi có thể triển khai tại các nút giao thông thực.
- **Tốc độ lưu thông:** Trong một số kịch bản, tốc độ trung bình của DQN thấp hơn Fixed-Time do cách đo lường bao gồm cả xe đang dừng chờ pha. Cần nghiên cứu thêm chỉ số này.

5.3.2 Hướng phát triển tương lai

- **Multi-Agent Reinforcement Learning (MARL):** Mở rộng từ điều khiển một nút giao lên điều phối đồng thời nhiều nút trong một mạng lưới giao thông, sử dụng các agent học cách phối hợp với nhau để tối ưu luồng xe trên toàn tuyến.
- **Transfer Learning:** Tận dụng trọng số model đã huấn luyện (`dqn_vn_tls.pt`) làm điểm khởi đầu (pretrained model) cho các kịch bản mới, giảm đáng kể thời gian huấn luyện từ đầu.
- **Tích hợp dữ liệu thực tế:** Thay thế phân bố xe ngẫu nhiên bằng dữ liệu lưu lượng giao thông thực tế từ camera hoặc cảm biến tại các nút giao thực sự ở Hà Nội.

- **Phần thưởng nâng cao:** Bổ sung vào hàm reward các yếu tố như mức độ ô nhiễm khí thải, tiêu hao nhiên liệu và mức độ an toàn giao thông để tối ưu hóa toàn diện hơn.
- **Triển khai nhúng (Edge Deployment):** Nghiên cứu nén mô hình (model pruning, quantization) để có thể chạy trực tiếp trên phần cứng nhúng tại nút đèn tín hiệu với tài nguyên tính toán hạn chế.

Tài liệu tham khảo

References

[1] R. Bellman, *"A Markovian Decision Process,"* Journal of Mathematics and Mechanics, vol. 6, no. 5, pp. 679–684, 1957.

[2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.

[3] V. Mnih, K. Kavukcuoglu, D. Silver et al., *"Human-level control through deep reinforcement learning,"* Nature, vol. 518, pp. 529–533, 2015.

[4] H. van Hasselt, A. Guez, and D. Silver, *"Deep Reinforcement Learning with Double Q-learning,"* in *Proc. 30th AAAI Conference on Artificial Intelligence (AAAI)*, Phoenix, AZ, 2016, pp. 2094–2100.

[5] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, *"Dueling Network Architectures for Deep Reinforcement Learning,"* in *Proc. 33rd International Conference on Machine Learning (ICML)*, New York, NY, 2016, pp. 1995–2003.

[6] P. A. Lopez, M. Behrisch, L. Bieker-Walz et al., *"Microscopic Traffic Simulation using SUMO,"* in *Proc. 21st IEEE International Conference on Intelligent Transportation Systems (ITSC)*, Maui, HI, 2018, pp. 2575–2582.

[7] H. Wei, G. Zheng, H. Yao, and Z. Li, *"IntelliLight: A Reinforcement Learning Approach for Intelligent Traffic Light Control,"* in *Proc. 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, London, UK, 2018, pp. 2496–2505.

[8] W. Genders and S. Razavi, *"Using a Deep Reinforcement Learning Agent for Traffic Signal Control,"* arXiv preprint arXiv:1611.01142, 2016.

[9] X. Liang, X. Du, G. Wang, and Z. Han, *"A Deep Reinforcement Learning Network for Traffic Light Cycle Control,"* IEEE Transactions on Vehicular Technology, vol. 68, no. 2, pp. 1243–1253, 2019.